Shri Vile Parle Kelavani Mandal's
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING
(Autonomous College Affiliated to the University of Mumbai)
NAAC Accredited with "A" Grade (CGPA: 3.18)

Academic Year:2025-26 SAP ID: 60003240268

DEPARTMENT OF INFORMATION TECHNOLOGY

COURSE CODE: DJS23ICH1501 **NAME: Mohammad Shaikh**

COURSE NAME: DevOps Laboratory **CLASS: T. Y. B. Tech**

BATCH: I2-2 **ROLL NO: I169**

EXPERIMENT NO. 6

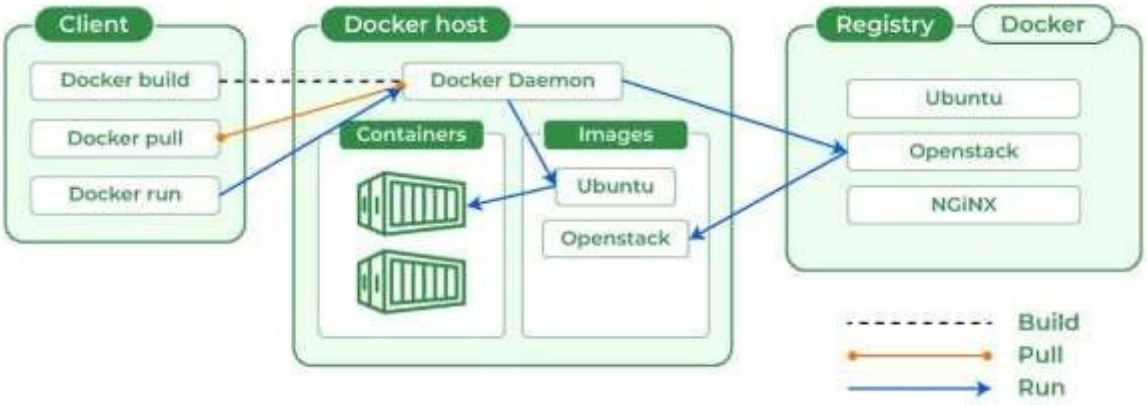
CO/LO: Apply DevOps principles to meet software development requirements.

AIM: To understand Docker Architecture and Container Life Cycle, install Docker and execute docker commands to manage images and interact with containers.

THEORY:

Docker Architecture:

Docker architecture. Docker uses a client-server architecture. The Docker client talks to the Docker daemon, which does the heavy lifting of building, running, and distributing your Docker containers. The Docker client and daemon can run on the same system, or you can connect a Docker client to a remote Docker daemon.



Docker Container Lifecycle Management

There are different stages when we create a Docker container which is known as Docker Container Lifecycle. Some of the states are:

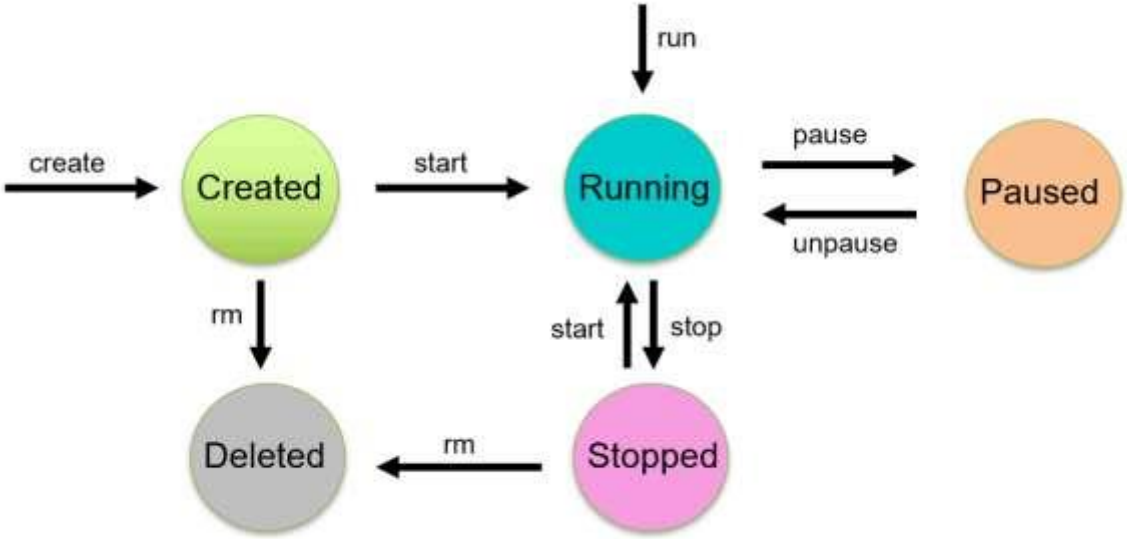
Created: A container that has been created but not started

Running: A container running with all its processes

Paused: A container whose processes have been paused

Stopped: A container whose processes have been stopped

Deleted: A container in a dead state



Project structure

devops-exp6/ ├─ app/
│ ├─ app.py
│ └─ requirements.txt
├─ Dockerfile
├─ docker-compose.yml
└─ lifecycle-demo.sh (bash script to demonstrate lifecycle)

app/app.py

```
from flask import Flask, jsonify
app =
Flask(__name__)

@app.route('/') def
hello():
    return jsonify({"message":"Hello from Flask in Docker!", "status":"ok"})
if __name__ == '__main__':
    # listen on 0.0.0.0 so container external port mapping works
app.run(host='0.0.0.0', port=5000)
```

app/requirements.txt *Flask==2.2.5*

Dockerfile

```
# Use a small Python image
FROM python:3.11-slim

# Create app dir
WORKDIR /app

# Copy requirements and install
COPY app/requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy application code COPY
app/ .

# Expose port
EXPOSE 5000

# Default command
CMD ["python", "app.py"]
```

docker-compose.yml

```
version:
'3.8'
services:
web:
build: .
    container_name: devops_exp6_web
ports:
      - "5000:5000"
restart: "no"
```

DEMO AUTOMATION SCRIPT (BASH)

lifecycle-demo.sh

```
#!/usr/bin/env bash set
-e

IMAGE=devops-exp6-web:1.0
NAME=lifecycle_demo_demo

echo "1) Creating container (not
starting)..." CID=$(docker create --name
$NAME $IMAGE) echo "Created container:
$CID" docker ps -a --filter "name=$NAME"
echo echo "2) Starting container..." docker
start
$NAME sleep 1
docker ps --filter "name=$NAME"
echo
echo "3) Show logs (brief)..."
docker logs --tail 10 $NAME || true
echo  echo "4) Pausing container (if
supported)..." if docker pause $NAME
2>/dev/null; then   echo "Paused."
```

```
  docker inspect -f '{{.State.Status}} Paused={{.State.Paused}}'
$NAME   sleep 2   docker unpause $NAME   echo "Unpaused." else
echo "Pause not supported on this platform." fi  echo echo "5) Stop
container..." docker stop $NAME docker ps -a --filter "name=$NAME"
echo echo "6) Remove container..." docker rm
$NAME
echo "Removed container."
 echo echo "7) Optionally remove image
(y/N)?" read -r -p "> " REPLY if [[ "$REPLY"
=~ ^([yY][eE][sS]|[yY])$ ]]; then   docker
rmi $IMAGE || true   echo "Image removed (if
existed)." else   echo "Image kept." fi   echo
"Done."
```

FINAL NOTES / TROUBLESHOOTING

- If docker pause fails, it's usually because host is Windows containers or pause unsupported — use Linux backend / WSL2.
- If docker run reports permission issues on Linux, make sure your user belongs to docker group or prefix commands with sudo.

- If port 5000 is already in use, use another host mapping -p 5001:5000.

OBSERVATIONS

Docker Installation:

Docker Engine/Docker Desktop was successfully installed and verified using the docker version and docker run hello-world commands. The test container confirmed that Docker is functioning correctly on the host system.

Image Creation:

A custom Docker image (devops-exp6-web:1.0) was successfully built using a Dockerfile. The image contained a simple Flask application and was visible in the local repository using docker images.

Container Lifecycle Management:

- The docker create command created a container in the Created state without running it.
- Using docker start, the container transitioned to the Running state and the application became accessible at http://localhost:5000/.
- docker pause and docker unpause correctly suspended and resumed container processes (on Linux backend).
- After executing docker stop, the container moved to the Exited/Stopped state.
- The container was removed completely using docker rm, showing the Deleted state in the lifecycle.
- Finally, the custom image was deleted using docker rmi.

Resource Usage:

Commands such as docker stats and docker inspect showed real-time resource utilization and container metadata, confirming Docker's efficient isolation and lightweight resource footprint.

Networking & Port Mapping:

The -p 5000:5000 option successfully mapped container ports to the host, enabling browserbased access to the Flask app from the local machine.

Consistency & Portability:

The same Docker image worked identically on different systems (Windows WSL2 and Linux), demonstrating Docker's portability and environment consistency.

CONCLUSION

The experiment successfully demonstrated the Docker architecture and container lifecycle — from image creation to container removal.

It was observed that Docker provides a lightweight and efficient alternative to traditional virtual machines by isolating applications within containers that share the host operating system kernel.

This exercise also reinforced understanding of key Docker concepts like images, containers, layers, networking, and port mapping.

Overall, Docker proves to be an essential DevOps tool that simplifies application deployment, ensures consistency across environments, and enables scalable, reproducible software delivery.